

8. Реализовать файловую систему для временного хранения данных. Файлы автоматически удаляются через заданный промежуток времени после создания.

```
#define FUSE_USE_VERSION 31
#include <fuse3/fuse.h>
#include <iostream>
#include <unordered_map>
#include <chrono>
#include <cstring>
#include <string>
#include <thread>

struct FileData {
    std::chrono::system_clock::time_point creation_time;
    int lifetime_seconds;
};

std::unordered_map<std::string, FileData> files;

int my_getattr(const char *path, struct stat *stbuf, struct fuse_file_info *fi) {
    std::cout << "Getting attributes for: " << path << std::endl;
    (void) fi;
    memset(stbuf, 0, sizeof(struct stat));

    if (strcmp(path, "/") == 0) {
        stbuf->st_mode = S_IFDIR | 0755;
        stbuf->st_nlink = 2;
    } else {
        auto it = files.find(path);
        if (it == files.end()) {
            return -ENOENT;
        }

        stbuf->st_mode = S_IFREG | 0644;
        stbuf->st_nlink = 1;
        stbuf->st_size = 1024;
    }

    return 0;
}

int my_open(const char *path, struct fuse_file_info *fi) {
    auto it = files.find(path);
    if (it == files.end()) {
        return -ENOENT;
    }

    return 0;
}
```

```

int my_read(const char *path, char *buf, size_t size, off_t offset, struct
fuse_file_info *fi) {
    auto it = files.find(path);
    if (it == files.end()) {
        return -ENOENT;
    }

    std::string content = "Temporary file content\n";
    size_t len = content.size();

    if (offset < len) {
        if (offset + size > len) {
            size = len - offset;
        }
        memcpy(buf, content.c_str() + offset, size);
    } else {
        size = 0;
    }

    return size;
}

int my_create(const char *path, mode_t mode, struct fuse_file_info *fi) {
    std::cout << "Creating file: " << path << std::endl;

    FileData fd;
    fd.creation_time = std::chrono::system_clock::now();
    fd.lifetime_seconds = 10;

    files[path] = fd;

    return 0;
}

int my_unlink(const char *path) {
    auto it = files.find(path);
    if (it == files.end()) {
        return -ENOENT;
    }

    files.erase(it);
    return 0;
}

void cleanup_expired_files() {
    while (true) {
        auto now = std::chrono::system_clock::now();
        for (auto it = files.begin(); it != files.end(); ) {
            auto elapsed = std::chrono::duration_cast<std::chrono::seconds>(
                now - it->second.creation_time);

```

```

        if (elapsed.count() >= it->second.lifetime_seconds) {
            it = files.erase(it);
        } else {
            ++it;
        }
    }
    std::this_thread::sleep_for(std::chrono::seconds(1));
}
}

int my_readdir(const char *path, void *buf, fuse_fill_dir_t filler, off_t offset,
struct fuse_file_info *fi, enum fuse_readdir_flags flags) {
    std::cout << "Reading directory: " << path << std::endl;
    (void) offset;
    (void) fi;
    (void) flags;

    if (strcmp(path, "/") != 0) {
        return -ENOENT;
    }

    filler(buf, ".", NULL, 0, static_cast<fuse_fill_dir_flags>(0));
    filler(buf, "..", NULL, 0, static_cast<fuse_fill_dir_flags>(0));

    for (const auto &file : files) {
        filler(buf, file.first.c_str() + 1, NULL, 0,
static_cast<fuse_fill_dir_flags>(0));
    }

    return 0;
}

int my_utimens(const char *path, const struct timespec tv[2], struct
fuse_file_info *fi) {
    (void) fi;

    auto it = files.find(path);
    if (it == files.end()) {
        return -ENOENT;
    }

    if (tv != NULL) {
        it->second.creation_time = std::chrono::system_clock::now();
    }

    return 0;
}

static struct fuse_operations my_oper = {
    .getattr = my_getattr,
    .unlink = my_unlink,

```

```

        .open = my_open,
        .read = my_read,
        .write = NULL,
        .readdir = my_readdir,
        .create = my_create,
        .utimens = my_utimens
    };

int main(int argc, char *argv[]) {
    std::thread(cleanup_expired_files).detach();
    return fuse_main(argc, argv, &my_oper, NULL);
}

```

Результат работы

```

wwwxtus@DESKTOP-I1F97A5:~$ g++ -o lab3_3 lab3_3.cpp `pkg-config fuse3 --cflags --libs`
wwwxtus@DESKTOP-I1F97A5:~$ sudo ./lab3_3 lab3_temp/
wwwxtus@DESKTOP-I1F97A5:~$ sudo touch /lab3_temp
wwwxtus@DESKTOP-I1F97A5:~$ sudo ls /lab3_temp
/lab3_temp
wwwxtus@DESKTOP-I1F97A5:~$ echo "Hello, world!" | sudo tee /lab3_temp
Hello, world!
wwwxtus@DESKTOP-I1F97A5:~$ |

```

```

wwwxtus@DESKTOP-I1F97A5:~$ sudo ls lab3_temp
wwwxtus@DESKTOP-I1F97A5:~$ |

```